

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 唐菁 学号 2024K8009970006 专业 计算机科学与技术
实验项目编号 4-3 实验名称 复杂 IO 外设-深度学习算法与硬件加速器

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、基本思路说明、具体设计细节

1.1 底层 alu 和 custom_cpu 的实现

因为需要乘法, 所以要实现 MUL 指令。

首先在 `alu.v` 中实现乘法。考虑增加 ALUop 的位数从而增加操作的可能性。

最开始对乘法非常地简单:

```
1
2  assign mul_full = mul_A * mul_B;
3
4  // RV32 MUL: 只取低32位
5  assign mul_res = mul_full[31:0];
```

但是这随即导致 `bit_gen` 环节严重超时。考虑到这可能是因为乘法器, 消耗了大量的逻辑资源, 且其组合逻辑的路径极长, 而因为组合逻辑, 无论 `Aluop` 是什么, 都会进行这样的乘法运算, 导致即便在执行加减法等简单指令时, 整个 ALU 也必须等待耗时极长的乘法逻辑运算完成, 从而拉低了最高主频, 甚至导致布局布线工具无法满足时序约束。

所以, 我设置了这样一个三目运算符来选择 A 和 B, 从而让指令不是 MUL 的时候, 乘法器的输入直接是 0, 这样可以有效减少逻辑翻转, 并在一定程度上辅助综合工具进行逻辑优化。

从而在加减法的时候, 乘法器的功耗就不是这么大了, 从而成功通过了 `big_gen` 环节

```
1  assign mul_A = (ALUop == `MUL) ? (A) : 32'sd0;
2  assign mul_B = (ALUop == `MUL) ? (B) : 32'sd0;
3
4  assign mul_full = mul_A * mul_B;
5
6  // RV32 MUL: 只取低32位
7  assign mul_res = mul_full[31:0];
```

1.2 Custom_cpu 的设计

主要就是加入这一条指令的解码,然后 MUL 解码出来导致 Aluop 变成对应的乘法,然后让 alu 算出结果。

注意因为之前一些指令的解码并没有严格一一映射,导致有一些指令的解码会对 MUL 指令的 decode 造成覆盖和影响(见后续 debug)。这告诉我们,设计的时候应注意严谨性,否则可能对后续的扩展造成阻碍!

1.3 conv.c

这是高层的应用,也是关键。

先对卷积的逻辑进行阐释。卷积是一种对矩阵进行局部采样的一种方式,具有权重共享性。输入图片每一个通道对应卷积的一个通道,然后卷积核在输入图片上滑动,其每一个权值和对应该位置的像素值进行相乘,最后相加(不同通道也相加),得出输出矩阵上对应元素的值。而输出图片一共几个通道呢?就看看有几个卷积核,就分别对应几个通道。

其中,padding 的意思是,在输入矩阵周围进行填充,填充 0 等,从而增加输出图片的尺寸,也让边缘被充分地采样到。

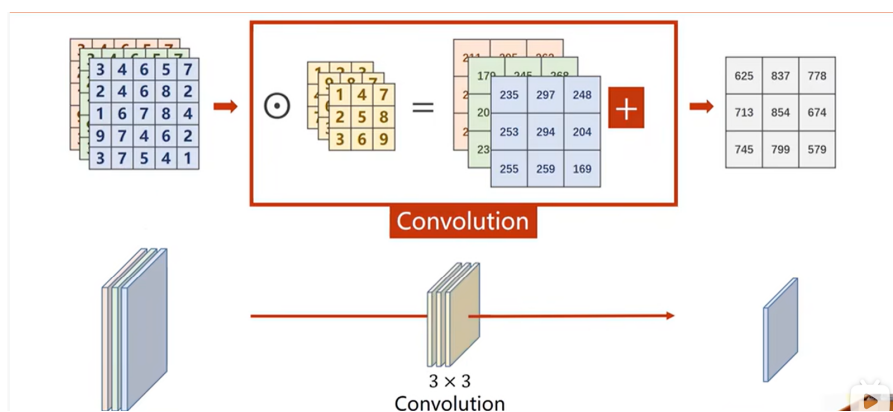


图 1: 卷积示意图 (图来自:bilibili 刘二大人-pytorch 深度学习实践)

接下来,对最大池化 max-pooling 地逻辑进行阐释。与卷积不同的是, max-pooling 只在高和宽的范围内进行相互作用,不同通道的像素不会相互作用。同样是一个“框”在图上滑动,这一通道的框内的最大值即为对应这一个通道的输出位置上的值。这相当于将整个图片中“最关键”的信息给出来了,能够抓取到图像的关键信息。

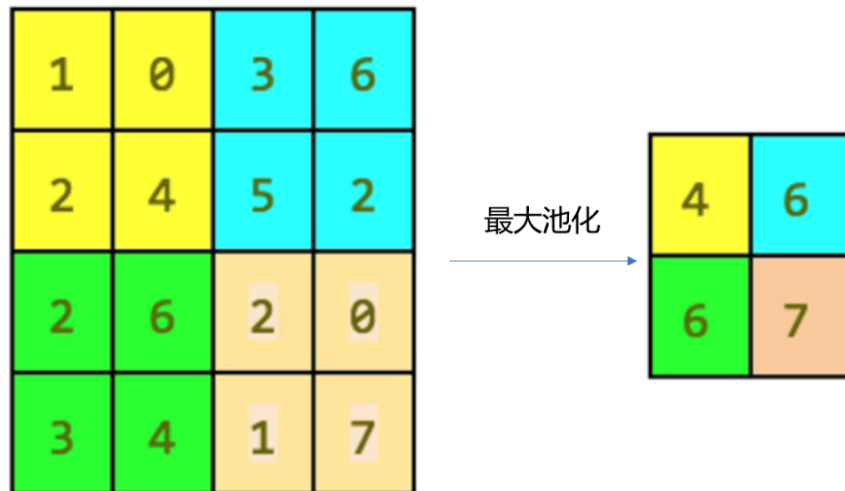


图 2: 最大池化示意图 (CSDN 博客- 【深度学习】 人人都能看得懂的卷积神经网络——入门篇-
<https://blog.csdn.net/fengdu78/article/details/106678697>)

然后,我们对具体代码的关键部分进行逐句阐释/公式提取和解读。

```

1 void convolution() {
2     //已经删除了非关键的代码
3
4
5     unsigned pad_len = pad << 1; //pad是两侧
6
7     unsigned conv_out_w = rd_size.d3 - weight_size.d3 + pad_len;
8     unsigned conv_out_h = rd_size.d2 - weight_size.d2 + pad_len;
9
10    unsigned stride = KERN_ATTR_CONV_STRIDE;
11
12    conv_out_w = div(conv_out_w, stride);
13    conv_out_h = div(conv_out_h, stride);
14
15    conv_out_w++;
16    conv_out_h++; //大小是(原来长度-卷积大小+padding乘以2)除以步长+1
17
18
19    unsigned elements_per_channel = 1 + Kh * Kw; //每一个通道的权重数目
20    unsigned elements_per_kernel = Ic * elements_per_channel; //每一个核所对应的权重数目
21
22    // 1. 遍历输出通道
23    for (unsigned no = 0; no < Oc; no++) {
24        // 取出当前输出通道的 bias
25        short bias_val = weight[no * elements_per_kernel];
26
27        for (unsigned y = 0; y < Oh; y++) {
28            for (unsigned x = 0; x < Ow; x++) {
29

```

```

30 // 将 bias 左移 10 位 (FRAC_BIT)，使其小数位从 10 位变成 20 位，与乘法结果对齐
31 long long sum = ((long long)bias_val) << FRAC_BIT;
32
33 for (unsigned ni = 0; ni < Ic; ni++) {
34     for (unsigned ky = 0; ky < Kh; ky++) {
35         for (unsigned kx = 0; kx < Kw; kx++) {
36
37             unsigned iw = kx + x * stride;
38             unsigned ih = ky + y * stride;
39
40             if ((iw >= pad) && (iw < input_fm_w + pad) &&
41                 (ih >= pad) && (ih < input_fm_h + pad)) {
42
43                 unsigned in_w = iw - pad;
44                 unsigned in_h = ih - pad;
45
46                 unsigned in_idx = ni * (input_fm_h * input_fm_w) + in_h * input_fm_w
47                     + in_w;
48                 //输入图像的位置
49                 // no * elements_per_kernel 定位到当前通道块
50                 // + 1 则是跳过开头的 bias 元素，进入权重区域
51
52                 unsigned weight_idx = no * elements_per_kernel + ni *
53                     elements_per_channel + 1 + ky * Kw + kx;
54                 //weight的位置
55                 sum += mul(in[in_idx], weight[weight_idx]);
56             }
57         }
58     }
59 }
60
61 // 四舍五入还原为 10 位小数
62 long long result_64 = sum >> FRAC_BIT;
63
64 // 饱和截断
65 if (result_64 > 32767) {
66     result_64 = 32767;
67 } else if (result_64 < -32768) {
68     result_64 = -32768;
69 }
70
71 unsigned out_idx = no * (Oh * Ow) + y * Ow + x;
72 out[out_idx] = (short)result_64;
73 }
74 }

```

计算的思路就是: 怎么计算? 是对 sum 的累加。对谁累加? 先对每一个输入通道进行遍历, 然后对卷积核滑

动到的那一块图像进行遍历。是卷积核对应的位置乘以图像对应的位置，累加。

第一步：确定输出目标。程序最外层遍历输出通道和输出平面的每个坐标点。对于每一个输出通道的每一个输出位置，首先加载偏置项 bias。为了保证精度，这里将 16 位的偏置左移了 FRAC_BIT 位，以便与后续乘法产生的结果在小数点位上对齐。

第二步：执行局部窗口内积。程序遍历所有输入通道，并在当前滑窗区域内进行卷积运算。它计算出输入图像的确切索引，如果该坐标在有效图像范围内，则将“图像像素”与“卷积核权重”相乘并累加到 sum 中。

第三步：精度还原与截断。当窗口内所有点及所有通道累加完成后，将结果右移还原回原来的定点小数位数。最后，通过一个饱和保护逻辑，将结果限制在 short 类型的表示范围(-32768 到 32767)之内，确保数据安全。然后我们来看 pool 的代码

```
1 void pooling()
2 //省略简单的赋值代码
3 {
4
5
6     for (unsigned c = 0; c < Channels; c++) //对每一个通道进行遍历
7     {
8         for (unsigned py = 0; py < Ph; py++) //每一个输出的height
9         {
10             for (unsigned px = 0; px < Pw; px++) //每一个输出的width坐标
11             {
12                 short max_val = -32768;
13
14                 for (unsigned ky = 0; ky < Kh; ky++) //每一个位置上的都是对应的框内的像素的最大值
15                 {
16                     for (unsigned kx = 0; kx < Kw; kx++)
17                     {
18                         unsigned ih = py * stride + ky;
19                         unsigned iw = px * stride + kx;
20
21                         unsigned in_idx =
22                             c * input_fm_h * input_fm_w +
23                             ih * input_fm_w +
24                             iw;
25
26                         short val = out[in_idx];
27
28                         if (val > max_val)
29                             max_val = val;
30                     }
31                 }
32
33                 unsigned out_idx =
34                     c * Ph * Pw +
35                     py * Pw +
36                     px;
37
38                 out[out_idx] = max_val;
39             }
9 }
```

```

40     }
41 }
42 }

```

思路就是对于每一个输出上的像素的值,都是对应的“框”内的像素的最大值。

1.4 硬件问询逻辑和比较逻辑

1. 问询逻辑

```

1  void launch_hw_accel()
2  {
3      volatile int* gpio_start = (void*)(GPIO_START_ADDR);
4      volatile int* gpio_done = (void*)(GPIO_DONE_ADDR);
5
6
7      // 1. 向 START 地址写入 1, 向硬件发出启动脉冲/信号
8      *gpio_start = 1;
9
10     // 2. 轮询: 死循环检查 DONE 地址的值
11     // 当硬件还在计算时, *gpio_done 为 0; 算完后硬件会把它置为 1
12     while (*gpio_done == 0) {
13         // 芯片会在这里挂起, 直到硬件加速器把活干完
14     }
15
16     // 3. 清除启动信号或中断, 为下一次运行做准备
17     *gpio_start = 0;
18
19
20 }
21 #endif

```

其中的逻辑是说: CPU 通知硬件加速器开始干活, 然后留在原地死等, 直到硬件加速器干完活, CPU 再继续往下执行。

为什么必须用 volatile 关键字? 代码中定义指针时使用了 volatile int*, 如果没有 volatile: 编译器在优化代码时, 会认为 while (*gpio_done == 0) 循环体内部没有修改 *gpio_done 的值。为了提高效率, 编译器可能会把这个值直接读入 CPU 寄存器, 然后变成死循环, 永远不会再去重新读取内存/寄存器的最新状态。

加上 volatile 后: 强制编译器每次循环都必须老老实实地去 GPIO_DONE_ADDR 这个物理地址重新读取数据。因为这个地址的值是由外部的硬件加速器修改的, CPU 必须实时监控它的变化。

2. comparing 逻辑

在没有硬件加速时, 对比的字节数就是标准答案的大小但在开启 USE_HW_ACCEL 后, 计算总数 count 变大了: 因为硬件加速器在处理内存数据时, 通常要求数据必须按照固定的宽度 (比如 32 字节) 对齐。如果原始数据的最后一部分没有填满这个宽度, 硬件就会在后面自动填充一些无用的字节。

结果硬件输出到 out 里的总字节数变成了 count, 因此 CPU 遍历 out 时必须扩大搜索范围, 否则会漏掉后面的有效数据。

所以, 由于 out 里面混入了硬件自动填充的垃圾数据, 而标准答案 result 里没有这些填充数据, 如果直接按顺序一一对比, 必然会错位报错。

代码通过下面这段逻辑跳过了这些硬件填充的字节:

```

1 int alignment = i & 0x0000001f; // 0x1f 即十进制的 31
2 if (alignment >= (WR_SIZE_D3 << 1))
3 continue;

```

其中, $i \& 0x0000001f$ 是位运算的取模操作, 等价于 $i \% 32$ 。这意味着硬件是以 32 字节为一个基本对齐周期。

过滤条件: 在每一个 32 字节的周期内, 前 $(WR_SIZE_D3 \ll 1)$ 字节是有效数据, 如果当前指针 i 在 32 字节周期内的偏移量超出了有效数据的范围, 说明当前字节是硬件为了凑齐 32 字节而填入的“垃圾数据”。此时, CPU 选择 `continue`, 不与标准答案比对。

二、 实验中遇到的 Bug 等问题的解决和思考

1. `bit_gen` 太慢了以至于超时:

方法是先注释了乘法器发现不超时, 然后开始思考如何优化乘法器逻辑。方法如上所述, 将乘法器输入变成 0 (当 `Aluop` 不是 `MUL`), 这样就可以减少逻辑反转, 从而减少这个问题的出现。

2. 当时连 `hardware` 都出错了, 开始思考是不是 `simple_cpu` 出了问题。思考发现 `alu` 没有问题, 那么只能是 `custom_cpu` 之中地问题了: 后来发现 `Instruction` 中:

```

1 assign ALUop =
2 (current_state == `IF || current_state == `IW)? `ADD: //算PC
3 (is_R && funct7 == 7'b0000001)? `MUL : //MUL //当还没有MUL提前写在这一行的时候, ((is_R || is_I)
   && funct3 == 3'b000)的ADD指令会将其覆盖。
4 (is_B && ((funct3 == 3'b110) || (funct3 == 3'b111) ) && (current_state == `EX)? `SLTU : // BGEU/BEU
5 (is_B && (funct3 == 3'b100 || funct3 == 3'b101) ) && (current_state == `EX)? `SLT : // BLT, BGE
6 (is_L || is_S || is_U_AUIPC)? `ADD : // 访存地址计算 / AUIPC
7 (is_B && (current_state == `ID)) ? `ADD : //算地址
8 (is_B && (current_state == `EX)) ? `SUB : //算地址
9 ((is_R || is_I) && funct3 == 3'b000) ? ((is_R && Ins[30]) ? `SUB : `ADD) : // add/sub
   (仅R型有sub)
10 ((is_R || is_I) && funct3 == 3'b010) ? `SLT : // slt
11 ((is_R || is_I) && funct3 == 3'b011) ? `SLTU : // sltu
12 ((is_R || is_I) && funct3 == 3'b100) ? `XOR : // xor
13 ((is_R || is_I) && funct3 == 3'b110) ? `OR : // or
14 ((is_R || is_I) && funct3 == 3'b111) ? `AND : // and
15
16 (is_J_JAL || is_J_JALR)? `ADD:
17 `ADD; // default, 这样平时PC+4也会合理

```

当 `MUL` 提前写在第二行的时候 (而是写在后面), $((is_R \parallel is_I) \&\& funct3 == 3'b000)$ 的 `ADD` 指令会将其覆盖, 从而造成软硬件全错...

3. 后来脑袋一晕, 听从了 AI 的建议, 将 `MUL` 指令的实现转成了 `signed`。这将出现问题, 导致硬件出错。因为在 Verilog 中, 只要表达式中有一个操作数是有符号数, 整个表达式的运算规则都会转变。这会直接破坏 ALU 内其他无符号指令的逻辑, 比如无符号比较 (如 `SLTU`) 直接失效: 原本 $A < B$ 是无符号比较, 一旦 A 和 B 变成 `signed`, 硬件会自动编译成有符号比较。

后来是发现硬件本来是对的, 后来错了, 比对文件发现关键点在这里, 细细思考之后得出, 其实不需要符号数。

4. `conv.c` 的问题倒是不多, 最多是写的时候存在一些逻辑上的错误或者语法错误, 透彻理解/仔细修改之后即可得到正确的答案。

5. 当时为了 debug, 让 AI 帮我修改了 conv.c 中 comparing 的逻辑。结果后来 hardware 一直出错, 报错如下:

```
1 Launching task..Failed
2 Byte Index=30
3 Out byteGold byeShort I5Out short=ffGold shortk=11 out=ff73 gofk=12 out=ff73 gofk=13 out=ff73
   gofk=14 out=ff73 gofk=15 out=ff73 go2k=16 out=ff73 go9k=17 out=ff73 godk=18 out=ff73 godk=19
   out=128 gol8bentime 1000657.49ms
```

首先, 我们可以发现错误的总是一样的字符。然后和之前 hardware 跑对的文件多次对比, 才意识到是 comparing 从跳过一些硬件的垃圾字符, 变成了比较所有的文字, 这时才会出现连填入的“垃圾字符”也被比较这件事。然后修改回去即可。这告诉我们用 AI 即使生成所谓的“调试信息”, 也应该谨慎思考和处理, 而非简单地替换, 否则 debug 起来很麻烦。

三、 对讲义中思考题(如有)的理解和回答

1. 如何表示十进制实数? 浮点数(IEEE-754): 但我们设计的 MIPS 处理器不支持浮点指令和处理部件

答: 直接用 16-bit 定点数表示, 用整数运算代替小数运算。这就是为什么偏置在相加的时候需要左移 10 位的原因: 因为相乘相当于左移了 20 位, 而偏置相当于仅仅左移了 10 位。为了小数点对齐, 所以需要偏置左移。这也是为什么最后要右移的原因: 还原为 10 位小数。

2. 在软件算法实现中, 需考虑如何避免出现溢出和精度损失-乘法、加法运算的中间结果可使用 32-bit 定点数来表示-请同学们思考如何扩展?

答: (1) 位宽扩展: 防止“乘法”阶段溢出在代码开头, 输入、权重和输出数组都被定义为 short 类型(即 16 位有符号整数):

```
1 Cshort *in = (short *)addr.rd\_addr;
2 short *weight = (short *)addr.weight\_addr;
```

但是在执行乘法时, 代码调用了 mul 函数, 其内部实现是:

```
1 mul(short a, short b) {
2     int ans = a * b; // 16位 * 16位 -> 隐式扩展为 32位 int
3     return ans;
4 }
```

两个 16 位有符号数相乘, 结果最大可能占用 32 位。这里在相乘时, C 语言会自动将操作数提升为 32 位 int 再进行乘法, 防止了乘法瞬间的高位截断溢出。

(2) 小数位对齐: 代码中定义了 #define FRAC_BIT 10, 这意味着数据 16 位总长度中, 有 10 位用来表示小数, 剩下的 6 位表示符号和整数。根据定点数运算法则: 乘法: $Q_{10} \times Q_{10} = Q_{20}$ (乘积拥有 20 位小数)。加法: 必须同阶对齐(即两数的小数位数必须相同)才能相加。即刚刚提到的对 bias 的处理。

(3) 超大容量累加器: 防止“大批量加法”溢出在卷积的内层循环中, 需要把一个通道内所有的乘法结果累加起来:

```
1 long long sum = ...; // 声明为 64 位有符号整数
2 ...
3 sum += mul(in[in_idx], weight[weight_idx]); // 64位 += 32位
```

一个 5×5 的卷积核要进行 25 次加法, 如果输入通道数很大, 加法迭代次数会非常多。32 位的 int 也有可能

在连续加法中溢出。因此,这里使用了 64 位的 long long sum 作为超大容量的累加器,绝对不会发生累加溢出。

(4) 饱和截断在把 64 位的 result_64 强制塞进 16 位的 short 之前,代码做了边界检查:

```
1
2 // 16位有符号数的范围是 -32768 到 32767
3 if (result_64 > 32767) {
4     result_64 = 32767; // 正向限幅
5 } else if (result_64 < -32768) {
6     result_64 = -32768; // 负向限幅
7 }
8 .out[out_idx] = (short)result_64; // 安全地强转为 16 位 short
```

如果计算出来的数太大(比如 32800),直接强转成 short 会变成负数,这在神经网络里会导致严重的计算错误。通过这种逻辑,如果数据超限,就让它死死卡在最大值 32767 或最小值 -32768,虽然损失了一点精度,但保证了硬件和算法的整体数值安全。

四、 实验所耗时间

在课后,你花费了大约 3 小时完成此次实验。